

TITLE: Driver Drowsiness Detection System

ABSTRACT

This project presents a real-time Driver Drowsiness Detection System (DDDS) aimed at enhancing road safety by alerting drivers when they exhibit signs of drowsiness or fatigue. Leveraging computer vision and machine learning techniques, the system monitors facial landmarks, particularly focusing on the eyes, to detect indicators of drowsiness such as eye closure. By calculating the Eye Aspect Ratio (EAR) and comparing it against a predefined threshold, the system can accurately determine the driver's level of alertness. Upon detecting drowsiness, the system issues timely alerts to the driver to mitigate the risk of accidents due to impaired vigilance. The effectiveness of the system is demonstrated through experimental evaluation using a webcam feed in real-world driving scenarios, showcasing its potential as a proactive safety measure on the roads.

திட்ட சுருக்கம்

இந்த திட்டம் நிகழ்நர ஓட்டுநர் தூக்கத்ததக் கண்டறிதல் அதமப்தப (DDDS) முன்தைக்கிற, இஃ ஓட்டுநர்கள் தூக்கம் அல்ல நஃோர்வு அறிகுறிகதள வைளிப்படுத்ம் நபோஃ எஃ்ரிக்தக வய்தைன் மூலம் ஃோதல போஃகோப்தப நமம்படுத்தை நஃோக்கமோகக் வகோண்டுள்ள. கணினி போர்தை மற்றும் இயந்திர கற்றல் நுட்பங்கதள நமம்படுத்தைன் மூலம், கணினி முக அதடயோளங்கதள கண்கோணிக்கிற, குறிப்போக கண்களில் கைனம் வயலுத்ஃகிற, கண் மூடல் நபோன்ற தூக்கத்தின் குறிகோட்டிகதளக் கண்டறியும். கண் போர்தை விகிதத்தத (EAR) கணக்கிட்டு முன் தைரயறுக்கப்பட்ட ரைம்புடன் ஒப்பிடுதைன் மூலம், இயக்கியின் விழிப்புணர்வின் அளதை கணினி ஃல்லியமோக தீர்மோனிக்க முடியும். உறக்கத்ததக் கண்டறிந்தம், கண்கோணிப்பு குதறபோடு கோரணமோக ஏற்படும் விபத்ஃகளின் அபோயத்ததத் தணிக்க, இயக்கிக்கு கணினி ஃரியோன நநரத்தில் எஃ்ரிக்தககதள ஃழங்குகிற. நிஜ-உலக ஓட்டுநர் கோட்சிகளில் வைப்நகம் ஊட்டத்ததப் பயன்படுத்தி நஃோததன மதிப்பீட்டின் மூலம் அதமப்பின் வயல்திறன் நிரூபிக்கப்படுகிற, ஃோதலகளில் ஒரு வயல்திறன்மிக்க போஃகோப்பு நடைடித்தகயோக அதன் திறதனக் கோட்டுகிற.

LIST OF FIGURES

FIG NO	FIGURES	PAGE NO
4.1	Use case Diagram	31
4.2	Activity Diagram	32
4.3	Class Diagram	33
6.1	Block Diagram	38
7.1	Landmarked Image of a person by Dlib	39
7.2	HELEN Dataset Samples	39
7.3	Eyes With Horizontal and Vertical Distance Marked For Eye Aspect Ratio Calculation.	40
8.1.1	Eye 1	45
8.1.2	Eye 2	45
8.1.3	Eye 3	45

TABLE OF CONTENTS

CHAPTER NO.	TITLE	PAGE NO.
	ACKNOWLEDGEMENT	i
	ABSTRACT ENGLISH	ii
	ABSTRACT TAMIL	iii
	LIST OF FIGURES	iv
1	INTRODUCTION	1
	1.1 WHAT IS MACHINE LEARNING?	1
	1.2 WHY USE COMPUTER VISION?	4
	1.3 PROJECT DESCRIPTION	6
2	SYSTEM ANALYSIS	8
	2.1 SYSTEM REVIEW	8
	2.2 LITERATURE SURVEY	8
	2.3 EXISTING SYSTEM	9
	2.4 PROPOSED SYSTEM	9
3	SYSTEM REQUIREMENTS	10
	3.1 HARDWARE REQUIREMENTS	10
	3.2 SOFTWARE REQUIREMENTS	10
	3.3 SOFTWARE DESCRIPTIONS	11
4	SYSTEM DESIGN	31
	4.1 USE CASE MODEL	31
	4.2 ACTIVITY DIAGRAM	32
	4.3 CLASS DIAGRAM	33
5	SYSTEM TESTING	34
	5.1 OBJECTIVES OF TESTING	34

	5.2 TESTING STRATEGIES	34
	5.2.1 White box testing	34
	5.2.2 Black box testing	34
	5.2.3 Top-Down testing	34
	5.2.4 Bottom-Up testing	35
	5.2.5 Unit Testing	35
	5.2.6 Integration Testing	36
	5.3 TEST CASES AND TEST RESULTS	37
6	PROJECT PLANNING	38
	6.1 SYSTEM MODEL	38
7	IMPLEMENTATION	39
8	SOURCE CODE	41
	8.1 ASSETS	45
9	SCREENSHOTS	47
	9.1 NON-DROWSY PERSON	47
	9.2 DROWSY PERSON	48
10	CONCLUSION	49
	REFERENCES	50

CHAPTER 1

INTRODUCTION

1.1 WHAT IS MACHINE LEARNING?

Machine learning (ML) is a field of study in artificial intelligence concerned with the development and study of statistical algorithms that can learn from data and generalize to unseen data, and thus perform tasks without explicit instructions.

Machine learning is a subset of artificial intelligence that involves the development of algorithms and statistical models that enable computers to learn and improve from experience without being explicitly programmed. It relies on data to identify patterns, relationships, and trends, which the algorithms then use to make predictions, decisions, or take actions.

Machine learning algorithms can be categorized into supervised learning, unsupervised learning, semi-supervised learning, and reinforcement learning, depending on the type of data they are trained on and the learning approach they use.

Here are some more aspects to explore about machine learning (ML):

1. Types of Machine Learning:

- **Supervised Learning:** The model is trained on a labeled dataset, where each example is associated with a target output. The model learns to map inputs to outputs.
- **Unsupervised Learning:** The model is trained on an unlabeled dataset, and it learns to find patterns or structure in the data without explicit guidance.
- **Semi-Supervised Learning:** A mix of labeled and unlabeled data is used for training.
- **Reinforcement Learning:** The model learns to make decisions by interacting with an environment. It receives feedback in the form of rewards or penalties based on its actions.

- **Deep Learning:** Deep neural networks with multiple layers are used to learn intricate patterns from data, often achieving state-of-the-art performance in various tasks like image recognition, natural language processing, and speech recognition.

2. Model Evaluation and Metrics:

- Various metrics are used to evaluate the performance of ML models, depending on the task. For classification tasks, metrics like accuracy, precision, recall, F1-score, and ROC-AUC are commonly used. For regression tasks, metrics like mean squared error (MSE), mean absolute error (MAE), and R-squared are used.

3. Feature Engineering:

- Feature engineering involves selecting, transforming, and creating features from raw data to improve the performance of ML models. It can include techniques like normalization, one-hot encoding, feature scaling, dimensionality reduction, and feature selection.

4. Hyperparameter Tuning:

- Hyperparameters are parameters that are set before the training process begins. Tuning these hyperparameters is essential for optimizing the performance of ML models. Techniques like grid search, random search, and Bayesian optimization are commonly used for hyperparameter tuning.

5. Model Deployment:

- Once a model is trained and evaluated, it needs to be deployed to production systems for making predictions on new data. Deployment involves considerations like scalability, latency, model monitoring, and version control.

6. Ethical Considerations:

- ML systems can inherit biases present in the data they are trained on, leading to unfair or discriminatory outcomes. Ensuring fairness, transparency, accountability, and privacy in ML systems is essential to mitigate these risks.

7. Challenges and Future Directions:

- Challenges in ML include issues like data scarcity, overfitting, interpretability, adversarial attacks, and ethical concerns. Ongoing research aims to address these challenges and advance the field of ML further.

Understanding these aspects of machine learning provides a comprehensive view of its capabilities, limitations, and potential applications in various domains.

Machine learning (ML) finds applications in numerous fields due to its ability to analyze large amounts of data, recognize patterns, and make predictions or decisions without being explicitly programmed for each task. Here are some prominent fields where ML is used and why:

- **Healthcare:** ML is used for medical image analysis, diagnosis prediction, personalized treatment recommendation, drug discovery, and patient monitoring. ML algorithms can analyze medical images such as MRI scans or X-rays to detect abnormalities or assist radiologists in diagnosis. They can also analyze patient data to predict disease risk or treatment outcomes.
- **Finance:** In finance, ML is used for fraud detection, risk assessment, algorithmic trading, credit scoring, and personalized financial services. ML algorithms can analyze large volumes of financial transactions to identify patterns indicative of fraudulent activity. They can also predict market trends and optimize trading strategies based on historical data.
- **Retail:** ML is used in retail for demand forecasting, customer segmentation, recommendation systems, price optimization, and supply chain management. ML algorithms can analyze customer data to identify segments with similar purchasing behavior and tailor marketing strategies accordingly. They can also analyze sales data to predict future demand and optimize inventory levels.
- **Transportation:** ML is used in transportation for route optimization, predictive maintenance, autonomous vehicles, traffic management, and ride-sharing optimization. ML algorithms can analyze traffic patterns to predict congestion and optimize route planning for vehicles. They can also analyze sensor data from vehicles to detect anomalies indicative of potential maintenance issues before they occur.
- **Manufacturing:** ML is used in manufacturing for predictive maintenance, quality control, supply chain optimization, process optimization, and product design. ML algorithms can analyze sensor data from manufacturing equipment to predict equipment failures and schedule maintenance proactively. They can also analyze production data to detect defects and optimize manufacturing processes.
- **Natural Language Processing (NLP):** ML is used in NLP for tasks such as sentiment analysis, language translation, text summarization, and chatbots. ML algorithms can analyze large volumes of text data to extract meaning, sentiment, and intent, enabling applications such as virtual assistants and language translation services.
- **Marketing:** ML is used in marketing for customer segmentation, personalized recommendations, churn prediction, and campaign optimization. ML algorithms can

analyze customer data to segment customers based on demographics, behavior, or preferences and tailor marketing campaigns accordingly. They can also predict which customers are likely to churn and take proactive measures to retain them.

Overall, ML is used across various fields because of its versatility, scalability, and ability to derive insights from data that were previously inaccessible using traditional methods.

1.2 WHY USE COMPUTER VISION?

Computer vision is used for a variety of purposes across different industries because of its ability to extract meaningful information from visual data. Some common reasons for using computer vision include:

1. Automation: Computer vision enables automation of tasks that traditionally require human intervention, such as quality control in manufacturing, object detection and tracking, and autonomous navigation.

2. Enhanced Understanding: It allows computers to understand and interpret the visual world, including images and videos, leading to applications such as facial recognition, object recognition, and scene understanding.

3. Efficiency: By automating visual tasks, computer vision can improve efficiency, reduce errors, and increase productivity in various fields, including healthcare, agriculture, retail, and security.

4. Insights from Visual Data: Computer vision algorithms can analyze large volumes of visual data to extract insights and patterns that may not be readily apparent to human observers. This can be valuable for tasks such as medical image analysis, environmental monitoring, and customer behavior analysis.

5. Innovation: Computer vision drives innovation by enabling the development of new technologies and applications, such as augmented reality, virtual reality, and smart cameras, which have the potential to transform various industries and improve people's lives.

Overall, computer vision offers a powerful set of tools and techniques for extracting knowledge and understanding from visual data, leading to a wide range of applications and benefits across different domains.

Some other Computer Vision uses:

1. **Complex Pattern Recognition:** Deep learning excels at automatically discovering intricate patterns and features within large datasets, making it highly effective for tasks such as image recognition, speech recognition, and natural language understanding.
2. **Hierarchical Feature Learning:** Deep learning models can learn hierarchical representations of data, extracting increasingly abstract and complex features as information passes through successive layers of the network. This capability allows for more nuanced and robust representations of data, leading to superior performance in many tasks.
3. **End-to-End Learning:** Deep learning enables end-to-end learning, where the entire system is trained jointly to optimize performance on a specific task. This eliminates the need for manual feature engineering, streamlining the development process and often resulting in more efficient and accurate models.
4. **Flexibility and Adaptability:** Deep learning models are highly flexible and adaptable to various types of data and tasks. They can be easily modified and fine-tuned to address different problems or accommodate changes in input data, making them versatile tools for diverse applications.
5. **Scalability:** Deep learning algorithms can scale effectively with the size of the data and computational resources available. By leveraging parallel computing and distributed training frameworks, deep learning models can handle massive datasets and complex computations, allowing for efficient training on large-scale problems.
6. **State-of-the-Art Performance:** Deep learning has consistently achieved state-of-the-art performance in a wide range of tasks, including image classification, object detection, machine translation, and speech recognition. Its ability to learn complex representations of data has led to breakthroughs in many fields, surpassing human-level performance in some cases.

7. **Automation and Efficiency:** Deep learning enables automation of tasks that were previously time-consuming or infeasible to perform manually. This includes tasks like image captioning, medical diagnosis, and autonomous driving, where deep learning models can analyze data and make decisions autonomously with minimal human intervention.
8. **Innovation and Research:** Deep learning has fueled significant advances in AI research and innovation, driving progress in areas such as reinforcement learning, generative modeling, and unsupervised learning. Its ability to learn from data has led to the development of novel algorithms and techniques that push the boundaries of what is possible in artificial intelligence.
9. **Real-World Applications:** Deep learning has been successfully applied to a wide range of real-world applications across various industries, including healthcare, finance, automotive, retail, and entertainment. From improving medical diagnosis to optimizing supply chain management, deep learning continues to create transformative impacts in numerous sectors.
10. **Continued Advancements:** The field of deep learning is continuously evolving, with ongoing research and development leading to new architectures, algorithms, and techniques. This continuous advancement ensures that deep learning remains at the forefront of AI innovation, driving further progress and breakthroughs in the years to come.

1.3 PROJECT DESCRIPTION

A computer vision system that can automatically detect driver drowsiness in a real-time video stream and then notify the alert if the driver appears to be drowsy.

Fatigue is a safety problem that has not yet been deeply tackled by any country in the world mainly because of its nature. Fatigue, in general, is very difficult to measure or observe unlike alcohol and drugs, which have clear key indicators and tests that are available easily. Probably, the best solutions to this problem are awareness about fatigue-related accidents and promoting drivers to admit fatigue when needed. The former is hard and much more expensive to achieve, and the latter is not possible without the former as driving for long hours is very lucrative.

Machine learning (ML) is used in eye detection for several reasons:

1. **Accuracy:** ML algorithms can be trained on large datasets of images containing eyes, allowing them to learn complex patterns and features that distinguish eyes from other objects or facial features. This can lead to highly accurate eye detection.
2. **Robustness:** ML algorithms can adapt to variations in lighting conditions, occlusions, and poses, making them robust in real-world scenarios where these factors can vary significantly.
3. **Efficiency:** Once trained, ML models can detect eyes quickly, making them suitable for real-time applications such as driver monitoring systems, gaze tracking in user interfaces, and facial recognition systems.
4. **Customization:** ML models can be customized and fine-tuned for specific applications or environments, allowing developers to optimize performance for their particular use case.
5. **Continuous Improvement:** ML models can be continuously improved by retraining them on new data, allowing them to adapt to changing conditions or to improve performance over time.

Overall, ML offers a powerful and flexible approach to eye detection that can meet the demands of various applications in fields such as computer vision, human-computer interaction, and biometrics.

CHAPTER 2

SYSTEM ANALYSIS

2.1 SYSTEM REVIEW

This survey is done to comprehend the need and prerequisite of the general population, and to do as such, we went through different sites and applications and looked for the fundamental data. Based on these data, we made an audit that helped us get new thoughts and make different arrangements for our task. We reached the decision that there is a need of such application and felt that there is a decent extent of progress in this field too.

2.2 LITERATURE SURVEY

1. Singh, S., Singla, P., Kaur, M. – “Journal of Intelligent Transportation Systems”

This paper provides an overview of the significance of detecting driver drowsiness in reducing road accidents. It discusses the physiological and behavioral signs of drowsiness in drivers and introduces the need for effective drowsiness detection systems.

2. Sullman, M.J.M., Stephens, A.N., Pajo, K. – “Accident Analysis & Prevention”

This review paper examines existing drowsiness detection systems, including both commercial products and academic prototypes. It analyzes the methodologies, sensors, and algorithms used in these systems, providing insights into their performance, limitations, and real-world applicability.

3. Kumar, M., Kumar, M., Kumar, V., Tiwari, S. – “IEEE Sensors Journal”

This paper explores different types of sensors used for drowsiness detection, such as eye-tracking sensors, physiological sensors, and vehicle-based sensors. It discusses the advantages and limitations of each sensor type and their suitability for detecting drowsiness in drivers.

4. Li, C.T., Lin, C.T., Chung, P.C., Wang, S.H. – “Expert Systems with Applications”

This study investigates the features extracted from sensor data for drowsiness detection, such as eye closure duration, blink rate, EEG power spectrum, and vehicle dynamics. It discusses different feature selection techniques used to identify relevant and discriminative features for accurate drowsiness detection.

5. Chi, T.H., Kim, J., Kim, J.W. – “Transportation Research Part C: Emerging Technologies”

This paper explores machine learning and statistical algorithms used for classifying drowsiness states. It discusses techniques such as Support Vector Machines, Neural Networks, and Hidden Markov Models, analyzing their suitability for real-time implementation and resource-constrained environments.

6. Sahayadhas, A., Sundaraj, K., Murugappan, M. – “Expert Systems with Applications”

This study examines the methodologies used to validate drowsiness detection systems, including data collection protocols, experimental setups, and performance metrics. It discusses performance evaluation metrics such as sensitivity, specificity, accuracy, and reaction time, providing insights into the reliability of these systems.

7. Dinges, D.F., Maislin, G., Brewster, R.M., Krueger, G.P. – “Sleep”

This paper discusses the challenges and limitations of existing drowsiness detection systems, such as robustness to varying lighting conditions, individual differences, and occlusions. It proposes potential solutions and future research directions to address these challenges, such as multimodal sensor fusion, adaptive algorithms, and personalized models.

2.3 EXISTING SYSTEM

Our current statistics reveal that just in 2023 in India alone, 155,622 persons were killed and 371,884 injured due to car related accidents. Of these, at least 35 percent were caused due to fatigue causing drivers to make mistakes.

Money motivates drivers to make unwise decisions like driving all night even with fatigue. This is mainly because the drivers are not themselves aware of the huge risk associated with driving when fatigued.

Some countries have imposed restrictions on the number of hours a driver can drive at a stretch, but it is still not enough to solve this problem.

2.4 PROPOSED SYSTEM

The best solutions to this problem is to creating a awareness about fatigue-related accidents and promoting drivers to admit fatigue when needed.

In our project we address a critical issue of driver fatigue and sleepiness leading to accidents on roads by developing a driver drowsiness detection system.

It continuously monitor the drivers eyes and give an alert to the driver if they are in fatigue and sleepiness state.

CHAPTER 3

SYSTEM REQUIREMENTS

3.1 HARDWARE REQUIREMENTS

Laptop with basic hardware - Used to run our code.

Webcam - Used to get the video feed.

The hardware requirements may serve as the basis for a contract for the implementation of the system and should therefore be a complete and consistent specification of the whole system. They are used by software engineers as the starting point for the system design. It should what the system does and not how it should be implemented.

RAM: 8 GB

GPU: AMD Radeon 4 GB

Processor: i3 Processor

Hard disk: 512 GB

3.2 SOFTWARE REQUIREMENTS

Python

Python 3 – Anaconda (Jupyter notebook)

Libraries

Numpy

Scipy

Playsound

Dlib

Imutils

opencv

Operating System

Windows or Ubuntu

3.3 SOFTWARE DESCRIPTIONS

3.3.1 Python

Python 3.12 – Anaconda (Jupyter notebook)

Python is the basis of the program that we wrote. It utilizes many of the python libraries.

3.3.2 Libraries

Numpy

NumPy is a fundamental package for scientific computing in Python. It provides support for large, multi-dimensional arrays and matrices, along with a collection of mathematical functions to operate on these arrays efficiently. Here are some key details about the NumPy package:

1. Arrays:

- NumPy's main object is the ndarray, a multi-dimensional array of elements of the same type.
- NumPy arrays can have any number of dimensions and are indexed by a tuple of non-negative integers.
- Arrays can be created from Python lists, tuples, or other iterable objects using the `numpy.array()` function.

2. Array Attributes:

- `ndarray.shape`: Returns the shape of the array (tuple of integers indicating the size of each dimension).
- `ndarray.ndim`: Returns the number of dimensions (or axes) of the array.
- `ndarray.size`: Returns the total number of elements in the array.
- `ndarray.dtype`: Returns the data type of the elements in the array.

3. Array Initialization:

- NumPy provides functions to create arrays with initial values, such as `numpy.zeros()`, `numpy.ones()`, `numpy.empty()`, `numpy.full()`, `numpy.arange()`, `numpy.linspace()`, etc.

4. Array Operations:

- NumPy provides a wide range of mathematical operations that can be performed directly on arrays without the need for explicit looping.
- Element-wise operations (addition, subtraction, multiplication, division, etc.) are performed efficiently with NumPy's broadcasting feature.
- NumPy also provides linear algebra operations (`numpy.dot()`, `numpy.linalg.inv()`, `numpy.linalg.solve()`, etc.) and statistical functions (`numpy.mean()`, `numpy.std()`, `numpy.median()`, etc.).

5. Indexing and Slicing:

- NumPy arrays support powerful indexing and slicing operations to access elements and subarrays.
- Slicing in NumPy arrays is similar to Python lists but can be applied to multiple dimensions.

6. Broadcasting:

- Broadcasting is a powerful mechanism in NumPy that allows arithmetic operations between arrays of different shapes.
- When operating on arrays with different shapes, NumPy automatically broadcasts the smaller array to the larger array's shape.

7. Vectorization:

- Vectorization is the process of converting an operation that would otherwise be performed on individual elements into a single operation that operates on entire arrays.
- Vectorized operations are typically more efficient than equivalent operations performed using explicit loops.

8. Random Number Generation:

- NumPy provides functions for generating random numbers from various probability distributions, such as `numpy.random.rand()`, `numpy.random.randn()`, `numpy.random.randint()`, `numpy.random.normal()`, etc.

9. Performance:

- NumPy's array operations are implemented in optimized C and Fortran code, making them much faster than equivalent operations implemented in pure Python.
- NumPy also provides tools for interfacing with libraries written in C, C++, or Fortran, allowing for further performance improvements if needed.

10. Integration with Other Libraries:

- NumPy is a foundational library for many other Python libraries used in scientific computing, data analysis, and machine learning, such as SciPy, pandas, Matplotlib, scikit-learn, TensorFlow, and PyTorch.

Overall, NumPy is essential for numerical computing in Python, providing a powerful array manipulation toolkit and efficient numerical operations for scientific computing applications.

NumPy can be utilized within the context of a driver drowsiness detection system:

1. Data Handling:

- NumPy arrays can efficiently store and manipulate sensor data, such as eye movement data from cameras or physiological signals from sensors (e.g., EEG, ECG).
- Sensor data can be represented as multi-dimensional arrays, and NumPy provides tools for loading, preprocessing, and transforming this data.

2. Feature Extraction:

- NumPy's array operations and mathematical functions can be used to extract relevant features from sensor data.
- For example, in vision-based drowsiness detection, NumPy can be used to perform operations like image filtering, edge detection, and feature extraction from eye images.

3. Signal Processing:

- NumPy provides tools for signal processing, such as Fourier transforms, filtering, and spectral analysis, which can be useful for processing physiological signals like EEG or ECG.
- Signal processing techniques implemented with NumPy can help extract informative features related to drowsiness from physiological data.

4. Machine Learning Integration:

- NumPy arrays are compatible with many machine learning libraries and frameworks used for building drowsiness detection models, such as scikit-learn, TensorFlow, or PyTorch.
- Sensor data processed and represented as NumPy arrays can be fed into machine learning algorithms for training and prediction.

5. Data Fusion:

- In some drowsiness detection systems, data from multiple sensors (e.g., vision-based and physiological sensors) may be fused to improve detection accuracy.
- NumPy can be used to integrate and process data from different sources, facilitating data fusion techniques.

6. Performance Optimization:

- NumPy's efficient array operations and broadcasting capabilities can help optimize the performance of data processing algorithms, making them suitable for real-time or near-real-time applications.
- Vectorized operations with NumPy can significantly speed up computations compared to equivalent operations implemented with pure Python loops.

7. Visualization:

- NumPy arrays can be easily visualized using libraries like Matplotlib, allowing developers to analyze and visualize sensor data during the development and evaluation of drowsiness detection algorithms.

While NumPy itself may not directly address the specific challenges of drowsiness detection, it serves as a foundational tool for handling, processing, and analyzing the data crucial for building effective drowsiness detection systems.

Scipy

SciPy is an open-source Python library used for scientific computing and technical computing. It builds upon the capabilities of NumPy and provides additional functionality for optimization, integration, interpolation, linear algebra, statistics, signal processing, image processing, and more. Here are some key details about the SciPy package:

1. Integration:

- SciPy provides functions for numerical integration, including single and multiple integration, adaptive quadrature, and integration of ordinary differential equations (ODEs).
- Integration functions include **quad()** for general integration, **dblquad()** and **tplquad()** for double and triple integration, and **odeint()** for solving ordinary differential equations.

2. Optimization:

- SciPy offers optimization algorithms for finding the minimum (or maximum) of functions, including unconstrained optimization, constrained optimization, and least-squares fitting.
- Optimization functions include **minimize()** for general unconstrained optimization, **minimize_scalar()** for scalar minimization, and **minimize_constrained()** for constrained optimization.

3. Interpolation:

- SciPy provides functions for interpolating data and constructing interpolating functions from scattered data points.
- Interpolation functions include **interp1d()** for one-dimensional interpolation, **griddata()** for interpolating scattered data on a grid, and **SmoothBivariateSpline()** for smoothing and interpolating 2D data.

4. Linear Algebra:

- SciPy extends NumPy's linear algebra capabilities by providing additional functions for solving linear systems, eigenvalue problems, and matrix decompositions.
- Linear algebra functions include **solve()** for solving linear systems, **eig()** and **eigh()** for computing eigenvalues and eigenvectors of square matrices, and **svd()** for computing the singular value decomposition of matrices.

5. Statistics:

- SciPy offers statistical functions for various probability distributions, hypothesis testing, and descriptive statistics.
- Statistical functions include **norm()** for normal distribution, **ttest_ind()** for independent t-tests, **chi2_contingency()** for chi-square tests, and **describe()** for summary statistics.

6. Signal Processing:

- SciPy provides tools for signal processing, including filtering, Fourier analysis, wavelet transforms, and spectral analysis.
- Signal processing functions include **convolve()** for convolution, **fft()** and **ifft()** for computing the discrete Fourier transform and its inverse, and **morlet()** for computing the Morlet wavelet.

7. Image Processing:

- SciPy includes functions for basic image processing tasks, such as reading and writing images, resizing, filtering, and morphological operations.
- Image processing functions include **imread()** and **imsave()** for reading and writing images, **resize()** for resizing images, and **convolve2d()** for 2D convolution.

8. Sparse Matrices:

- SciPy provides support for sparse matrices, which are useful for dealing with large matrices with mostly zero entries.

- Sparse matrix functions include **csr_matrix()** and **csc_matrix()** for creating compressed sparse row and compressed sparse column matrices, and various functions for matrix manipulation and arithmetic operations.

9. Integration with NumPy:

- SciPy seamlessly integrates with NumPy, allowing users to combine the array manipulation capabilities of NumPy with the advanced scientific computing functionality of SciPy.

Overall, SciPy is a powerful library for scientific computing in Python, offering a wide range of functions and tools for numerical computation, optimization, data analysis, and more. It is widely used in scientific research, engineering, data analysis, and machine learning applications.

some of SciPy's capabilities can be relevant to driver drowsiness detection:

1. Signal Processing:

- SciPy's signal processing module (**scipy.signal**) offers a variety of functions for filtering, spectral analysis, and other signal processing tasks.
- Filtering techniques can be used to preprocess sensor data, such as EEG signals or eye movement data, to remove noise and extract relevant features.
- Spectral analysis tools can be used to analyze frequency components in physiological signals, which may contain patterns indicative of drowsiness.

2. Image Processing:

- SciPy's image processing module (**scipy.ndimage**) provides functions for basic image processing tasks, such as filtering, morphological operations, and feature extraction.
- Vision-based drowsiness detection systems may utilize these functions to preprocess and analyze images captured by cameras, such as detecting facial landmarks or tracking eye movements.

3. Statistical Analysis:

- SciPy's statistics module (**scipy.stats**) offers a wide range of probability distributions, statistical tests, and descriptive statistics functions.

- Statistical analysis can be applied to physiological data or other sensor data to identify patterns or anomalies associated with drowsiness.
- Hypothesis tests provided by SciPy can be used to assess the significance of observed changes in driver behavior or physiological signals.

4. **Optimization:**

- SciPy's optimization module (**scipy.optimize**) provides algorithms for optimization and root-finding tasks.
- Optimization techniques can be used in various aspects of drowsiness detection systems, such as parameter tuning for machine learning models or optimizing feature extraction algorithms.

5. **Integration:**

- SciPy's integration module (**scipy.integrate**) offers functions for numerical integration and solving ordinary differential equations (ODEs).
- These functions can be used to model physiological processes related to drowsiness or to integrate dynamic models of driver behavior with sensor data.

6. **Linear Algebra:**

- SciPy's linear algebra module (**scipy.linalg**) provides additional functionality for linear algebra operations, such as matrix decompositions and solving linear systems.
- Linear algebra techniques can be applied in various aspects of data analysis and modeling within drowsiness detection systems.

Its rich collection of scientific computing capabilities can be invaluable for processing, analyzing, and modeling the data involved in such systems. Integrating SciPy with other libraries and frameworks, such as NumPy, scikit-learn, and OpenCV, can further enhance the capabilities of drowsiness detection systems.

Dlib

Dlib is a versatile open-source library primarily written in C++, but with Python bindings, designed for machine learning, computer vision, and image processing tasks. It offers a wide range of functionalities that can be utilized in various applications, including driver drowsiness detection systems. Here are some key details about the Dlib library:

1. Facial Landmark Detection:

- One of the most notable features of Dlib is its facial landmark detection functionality. It can detect and localize key facial landmarks such as eyes, eyebrows, nose, mouth, and jawline in images or video frames.
- Facial landmark detection is crucial for tracking eye movements, which is a common approach in driver drowsiness detection systems.

2. Facial Expression Recognition:

- Dlib provides tools for facial expression recognition, allowing developers to detect facial expressions such as happiness, sadness, anger, and surprise.
- Facial expression analysis can be used as an additional input for drowsiness detection systems to infer the driver's emotional state and level of alertness.

3. Machine Learning Algorithms:

- Dlib includes implementations of various machine learning algorithms, such as support vector machines (SVM), k-nearest neighbors (KNN), and ensemble classifiers.
- These algorithms can be used for classification tasks related to drowsiness detection, such as distinguishing between alert and drowsy states based on features extracted from facial landmarks or physiological signals.

4. Object Detection and Tracking:

- Dlib offers object detection and tracking capabilities, which can be useful for detecting other objects in the driver's environment, such as vehicles, pedestrians, or road signs.
- Object detection and tracking can provide additional context for drowsiness detection systems and help identify potential hazards.

5. Image Processing Utilities:

- Dlib provides a variety of image processing utilities for tasks such as image resizing, rotation, cropping, and filtering.
- These utilities can be used for preprocessing images or video frames before applying drowsiness detection algorithms.

6. Integration with OpenCV:

- Dlib seamlessly integrates with OpenCV, another popular computer vision library in Python. Developers can leverage the functionalities of both libraries together to build robust drowsiness detection systems.
- OpenCV can be used for tasks such as capturing video frames from cameras, image preprocessing, and visualization, while Dlib can handle more complex tasks like facial landmark detection and expression recognition.

7. Cross-Platform Compatibility:

- Dlib is designed to be cross-platform and can run on various operating systems, including Windows, macOS, and Linux.
- This cross-platform compatibility makes it suitable for developing drowsiness detection systems that can be deployed on different types of devices and platforms.

Overall, Dlib offers a comprehensive set of tools and algorithms that can be valuable for building driver drowsiness detection systems. Its facial landmark detection, expression recognition, and machine learning capabilities, coupled with its integration with OpenCV, make it a powerful library for computer vision tasks in this domain.

Dlib is a versatile library that provides various tools and functionalities useful for driver drowsiness detection systems. Here's how specific features of Dlib can be applied in this context:

1. Facial Landmark Detection:

- Dlib's facial landmark detection feature is particularly useful for tracking eye movements, which is a key indicator of drowsiness. By detecting landmarks such as the corners of the eyes, eyebrows, and mouth, Dlib can track changes in facial expressions and movements indicative of drowsiness.

2. Real-Time Processing:

- Dlib's efficient algorithms allow for real-time processing of video streams from cameras, making it suitable for implementation in driver monitoring systems where timely detection of drowsiness is crucial for safety.

3. Eye Aspect Ratio (EAR) Calculation:

- Dlib can calculate the Eye Aspect Ratio (EAR), a metric commonly used in drowsiness detection algorithms. EAR is calculated based on the ratio of distances between facial landmarks around the eyes, and it decreases as the eyes close, indicating drowsiness.

4. Machine Learning Models:

- Dlib provides pre-trained models for facial landmark detection and facial expression recognition, which can be utilized to detect signs of drowsiness based on facial features.
- Additionally, Dlib's machine learning capabilities can be employed to train custom models on labeled datasets specific to drowsiness detection tasks.

5. Integration with OpenCV:

- Dlib can seamlessly integrate with OpenCV for tasks such as capturing video frames from cameras, preprocessing images, and visualizing detection results.
- OpenCV's functionalities complement Dlib's capabilities, enabling a comprehensive solution for driver drowsiness detection.

6. Customization and Development:

- Dlib is open-source and allows for customization and extension according to specific requirements of drowsiness detection systems.
- Developers can leverage Dlib's APIs and documentation to tailor algorithms and functionalities to the unique needs of their applications.

7. Cross-Platform Compatibility:

- Dlib is designed to be cross-platform, ensuring compatibility with various operating systems and hardware configurations commonly found in automotive environments.
- This compatibility enables the deployment of drowsiness detection systems across different platforms and devices.

By leveraging Dlib's features for facial landmark detection, real-time processing, machine learning, and integration with OpenCV, developers can build robust and efficient

driver drowsiness detection systems capable of accurately identifying signs of fatigue or distraction in drivers.

Imutils

Imutils is a convenience library built upon OpenCV, primarily designed to simplify common image processing tasks and streamline the development process. It provides a collection of functions that extend the functionality of OpenCV, making it easier to perform various image processing operations. Here are some key details about the Imutils package:

1. Image Resizing and Scaling:

- Imutils provides functions for resizing and scaling images to specific dimensions or aspect ratios.
- The **resize()** function allows for resizing images while preserving the aspect ratio, ensuring that the content of the image remains undistorted.

2. Translation and Rotation:

- Imutils offers functions for translating (shifting) and rotating images.
- The **translate()** function can be used to shift the position of an image horizontally or vertically by a specified number of pixels.
- The **rotate()** function allows for rotating images by a specified angle, with options for specifying the center of rotation and whether to crop or pad the image to maintain its original size.

3. Image Display:

- Imutils includes functions for displaying images in a window or saving them to disk.
- The **imshow()** function displays an image in a window with the specified window title.
- The **cv2.imshow()** function from OpenCV is wrapped by Imutils to simplify its usage.

4. Convenience Functions:

- Imutils provides several convenience functions for working with images and coordinates.
- Functions like **translate_contour()** and **rotate_bound()** extend OpenCV's functionality for translating and rotating image contours and bounding boxes.

5. Annotation and Drawing:

- Imutils offers functions for annotating images with text and drawing shapes like lines, rectangles, circles, and polygons.
- These functions can be used to highlight regions of interest or visualize results of image processing algorithms.

6. Video Stream Processing:

- Imutils includes utilities for working with video streams, such as reading frames from a video file or webcam and writing frames to a video file.
- The **VideoStream** class provides a convenient interface for capturing frames from a webcam in real-time.

7. Cross-Platform Compatibility:

- Imutils is compatible with various platforms and environments, including Windows, macOS, and Linux.
- It seamlessly integrates with OpenCV, allowing for easy adoption in existing OpenCV-based projects.

8. Code Simplification:

- Imutils simplifies common image processing tasks by providing easy-to-use functions and utilities.
- It reduces the amount of boilerplate code required to perform tasks like image resizing, rotation, and annotation.

Overall, Imutils is a valuable tool for simplifying image processing workflows and accelerating the development of computer vision applications, including driver drowsiness detection systems. Its collection of functions and utilities can help streamline tasks such as

image preprocessing, annotation, and visualization, ultimately leading to more efficient and maintainable code.

Imutils can be applied in driver drowsiness detection:

1. Facial Landmark Detection:

- Imutils can be used in conjunction with facial landmark detection algorithms (e.g., using Dlib or OpenCV) to preprocess and manipulate images or video frames containing driver's faces.
- Functions for resizing, rotating, or translating images provided by Imutils can help ensure that the detected facial landmarks are aligned properly for further analysis.

2. Image Resizing and Scaling:

- Imutils offers functions for resizing and scaling images, which can be useful for standardizing input images from different sources or cameras.
- Resizing images to a consistent size can improve the performance of facial landmark detection algorithms and other computer vision tasks.

3. Real-Time Processing:

- Imutils can facilitate real-time processing of video streams from cameras, which is essential for implementing driver monitoring systems that continuously analyze driver behavior.
- Functions for efficiently reading frames from a video stream and performing operations on them can be beneficial for timely detection of drowsiness.

4. Annotation and Visualization:

- Imutils provides utilities for annotating images with text and drawing shapes, which can be useful for visualizing the results of drowsiness detection algorithms.
- Annotations can help highlight regions of interest in images, such as detected facial landmarks or regions indicative of drowsiness.

5. Integration with OpenCV:

- Imutils seamlessly integrates with OpenCV, a widely-used computer vision library, enhancing its functionality and ease of use.
- OpenCV offers a wide range of algorithms and techniques that can be applied in driver drowsiness detection, and Imutils complements these capabilities with its convenient utilities.

6. Cross-Platform Compatibility:

- Imutils is compatible with various operating systems and environments, ensuring that driver drowsiness detection systems built with it can be deployed across different platforms.
- This compatibility makes it easier to develop portable and scalable solutions for drowsiness detection in diverse automotive environments.

Its collection of image processing utilities and seamless integration with OpenCV can still be valuable assets in developing efficient and effective drowsiness detection systems. By leveraging Imutils alongside other computer vision tools and algorithms, developers can streamline the development process and enhance the performance of their drowsiness detection solutions.

Opencv

OpenCV (Open Source Computer Vision Library) is a powerful open-source computer vision and machine learning software library. It offers a vast array of functionalities for image and video processing, object detection, feature extraction, and machine learning algorithms. Here are some key details about the OpenCV library:

1. Image Processing:

- OpenCV provides a wide range of image processing functions, including filtering, edge detection, morphological operations, histogram equalization, color space conversion, and image transformation.
- These functions allow for tasks such as noise reduction, feature extraction, image enhancement, and geometric transformations.

2. Video Processing:

- OpenCV supports video I/O operations, enabling the capture, processing, and playback of video streams from various sources such as cameras, files, or network streams.
- Video processing functionalities include frame-by-frame analysis, motion detection, object tracking, and video compression.

3. Object Detection and Tracking:

- OpenCV includes implementations of popular object detection and tracking algorithms, such as Haar cascades, HOG (Histogram of Oriented Gradients), and deep learning-based detectors (e.g., YOLO, SSD).
- These algorithms enable tasks like face detection, pedestrian detection, vehicle detection, and object tracking in videos.

4. Feature Detection and Description:

- OpenCV offers algorithms for detecting and describing image features, such as corners, blobs, and keypoints.
- Feature detection and description are essential for tasks like image matching, object recognition, and image registration.

5. Machine Learning:

- OpenCV provides machine learning algorithms and tools for tasks like classification, regression, clustering, and dimensionality reduction.
- It includes implementations of popular machine learning algorithms such as support vector machines (SVM), k-nearest neighbors (KNN), decision trees, and random forests.

6. Deep Learning Integration:

- OpenCV integrates with deep learning frameworks like TensorFlow and PyTorch, allowing users to deploy and run pre-trained deep learning models for tasks like image classification, object detection, and image segmentation.
- It provides functions for loading, preprocessing, and post-processing deep learning models, making it easier to integrate them into computer vision applications.

7. Camera Calibration and 3D Reconstruction:

- OpenCV includes tools for camera calibration, stereo vision, and 3D reconstruction from multiple images.
- These functionalities are used in applications like 3D object modeling, augmented reality, and structure-from-motion.

8. Cross-Platform Compatibility:

- OpenCV is compatible with various operating systems, including Windows, macOS, Linux, Android, and iOS.
- It supports multiple programming languages, including C++, Python, Java, and MATLAB, making it accessible to a broad community of developers.

9. Community and Documentation:

- OpenCV has a vibrant community of developers and researchers who contribute to its development and provide support through forums, mailing lists, and documentation.
- It offers comprehensive documentation, tutorials, and examples to help users get started with using the library.

Overall, OpenCV is a comprehensive and versatile library for computer vision and machine learning tasks, offering a rich set of functionalities and tools for developing a wide range of applications, including driver drowsiness detection systems. Its extensive feature set, cross-platform compatibility, and active community support make it a popular choice for researchers, engineers, and developers in the field of computer vision.

OpenCV (Open Source Computer Vision Library) offers various functionalities and tools that can be leveraged in the development of driver drowsiness detection systems. Here's how specific features of OpenCV can be applied in this context:

1. Face Detection:

- OpenCV provides pre-trained Haar cascades and deep learning-based models for detecting faces in images and video streams.
- Face detection is a crucial step in driver drowsiness detection systems as it allows for the localization of the driver's face in the camera feed.

2. Facial Landmark Detection:

- OpenCV can be used in conjunction with libraries like Dlib or facial landmark detection algorithms to detect key facial landmarks such as eyes, eyebrows, nose, and mouth.
- Detecting facial landmarks enables the measurement of eye movements, which is a key indicator of drowsiness.

3. Eye Tracking:

- By detecting and tracking the position of the driver's eyes using OpenCV, it's possible to monitor eye movements and analyze patterns associated with drowsiness, such as eyelid closure and eye gaze direction.
- Techniques like optical flow and template matching can be applied to track eye movements in real-time.

4. Image Preprocessing:

- OpenCV offers a wide range of image processing functions for preprocessing camera frames before drowsiness detection analysis.
- Operations such as image resizing, color space conversion, noise reduction, and contrast enhancement can improve the accuracy and reliability of drowsiness detection algorithms.

5. Real-Time Processing:

- OpenCV's efficient algorithms and optimized implementations enable real-time processing of video streams captured from cameras.
- Real-time processing is essential for continuously monitoring driver behavior and providing timely alerts or interventions in the event of drowsiness.

6. Alert Mechanisms:

- OpenCV can be integrated with audio and visual alert mechanisms to notify the driver or trigger automated responses when signs of drowsiness are detected.
- For example, alerts can be generated through sound alarms, flashing lights, or haptic feedback to wake up the driver or prompt them to take corrective action.

7. Integration with Machine Learning Models:

- OpenCV seamlessly integrates with machine learning frameworks like scikit-learn, TensorFlow, and PyTorch, allowing for the deployment of custom machine learning models for drowsiness detection.
- Machine learning models can be trained on labeled datasets to recognize patterns indicative of drowsiness in driver behavior or facial expressions.

8. Cross-Platform Compatibility:

- OpenCV is compatible with various platforms and operating systems, including Windows, macOS, Linux, Android, and iOS.
- This compatibility enables the deployment of drowsiness detection systems across different types of vehicles and hardware configurations.

3.3.3 Operating System

Windows 8

The primary requirement is a Windows 8 operating system installed on the target device. This includes both the desktop and tablet versions of Windows 8.

3.3.4 Visual Studio

The primary requirement is Visual Studio, which is an integrated development environment (IDE) developed by Microsoft. Visual Studio provides a comprehensive set of tools for software development, including code editing, debugging, testing, and project management.

3.3.5 Technology Used

a. PYTHON - Python is an interpreted, high-level, general-purpose programming language. Python's design philosophy emphasizes code readability with its notable use of significant whitespace. Its language constructs and object-oriented approach aim to help programmers write clear, logical code for small and large-scale projects. Python is dynamically typed AND supports multiple programming paradigms, including procedural, object-oriented, and functional programming.

b. JUPYTER Lab - Project Jupyter is a nonprofit organization created to develop open-source software, open-standards, and services for interactive computing across dozens of programming languages.

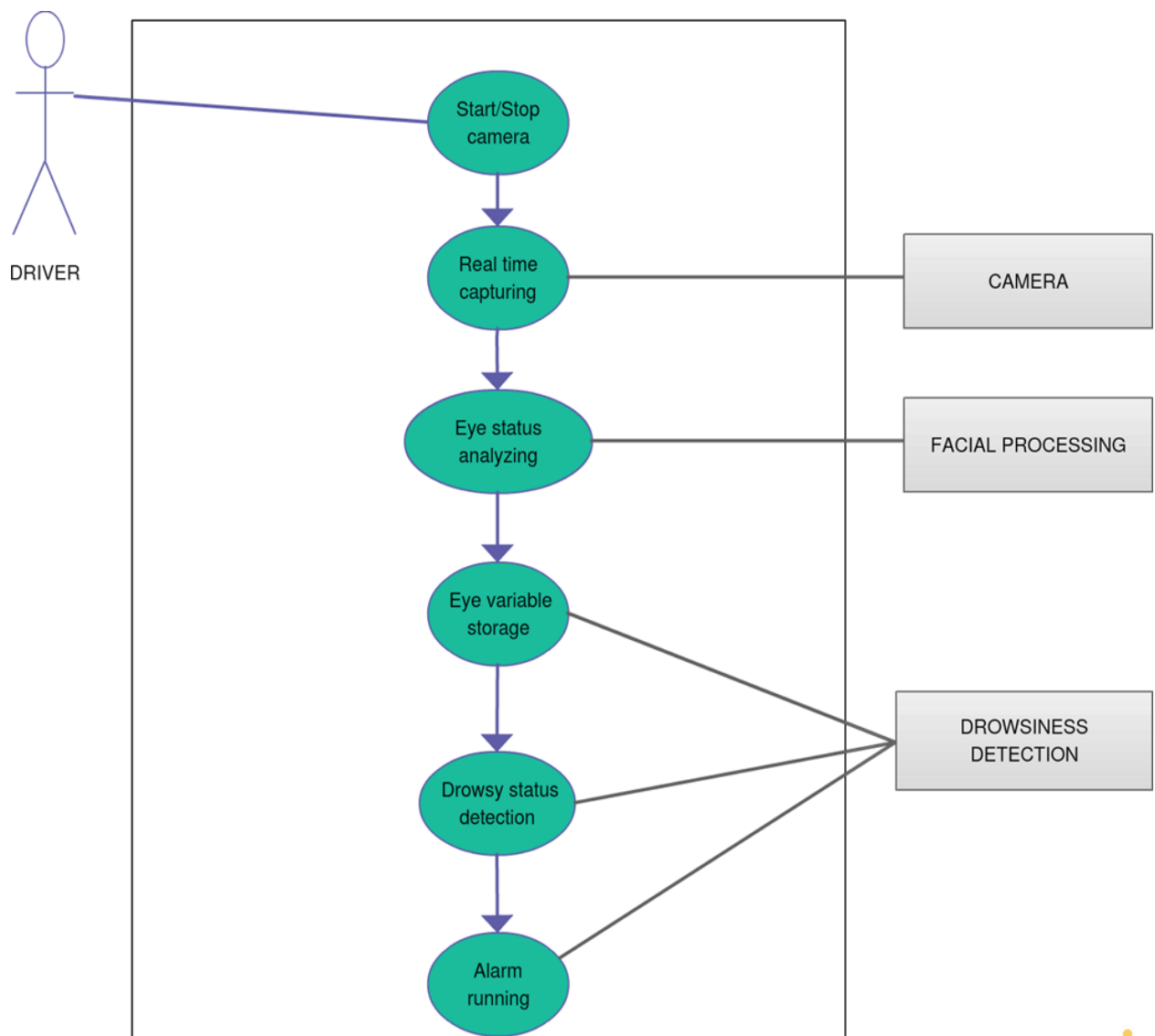
c. IMAGE PROCESSING - In computer science, digital image processing is the use of computer algorithms to perform image processing on digital images.

d. MACHINE LEARNING - Machine learning is the scientific study of algorithms and statistical models that computer systems use in order to perform a specific task effectively without using explicit instructions, relying on patterns and inference instead. It is seen as a subset of artificial intelligence. Machine learning algorithms build a mathematical model based on sample data, known as "training data", in order to make predictions or decisions without being explicitly told.

CHAPTER 4

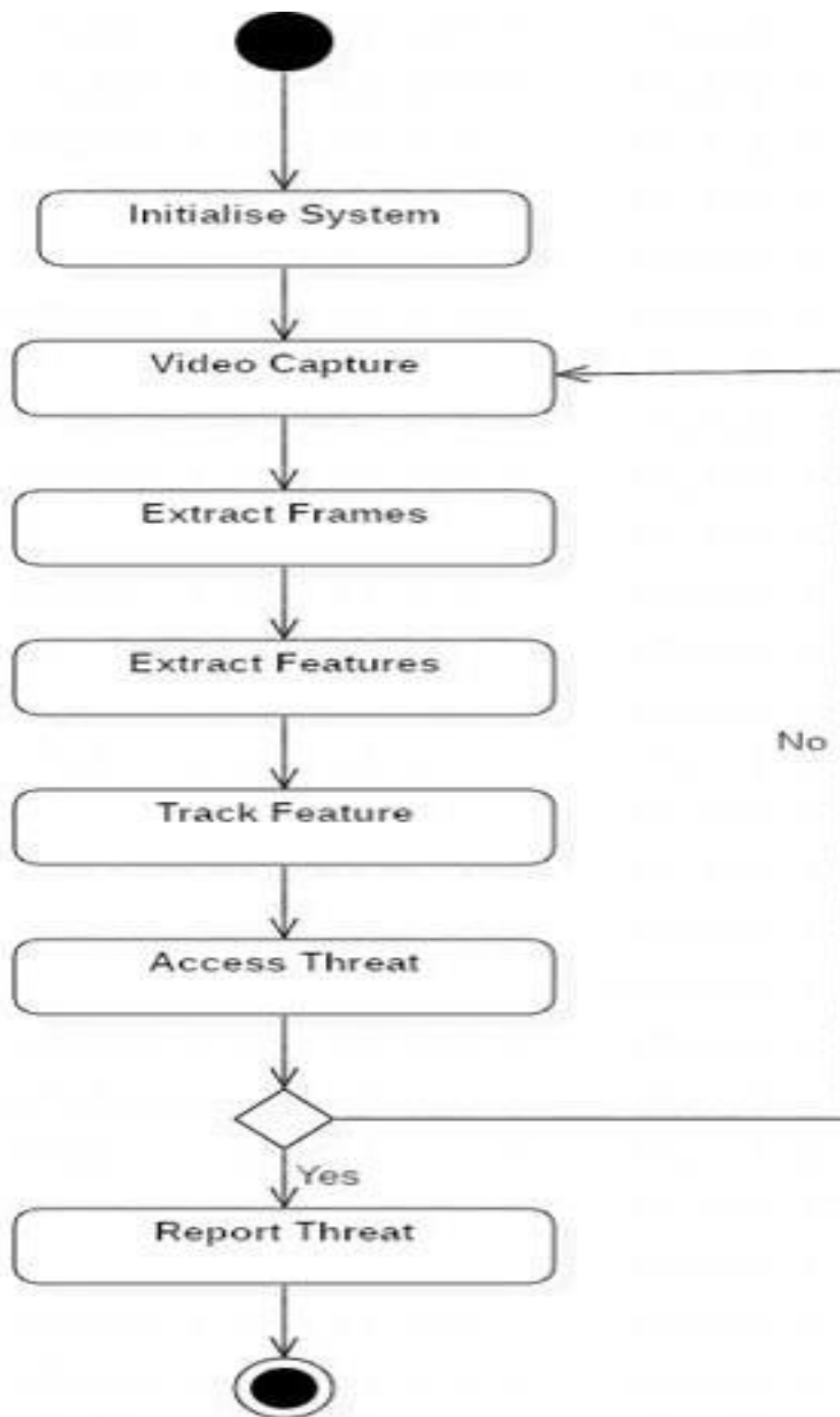
SYSTEM DESIGN

4.1 USE CASE MODEL



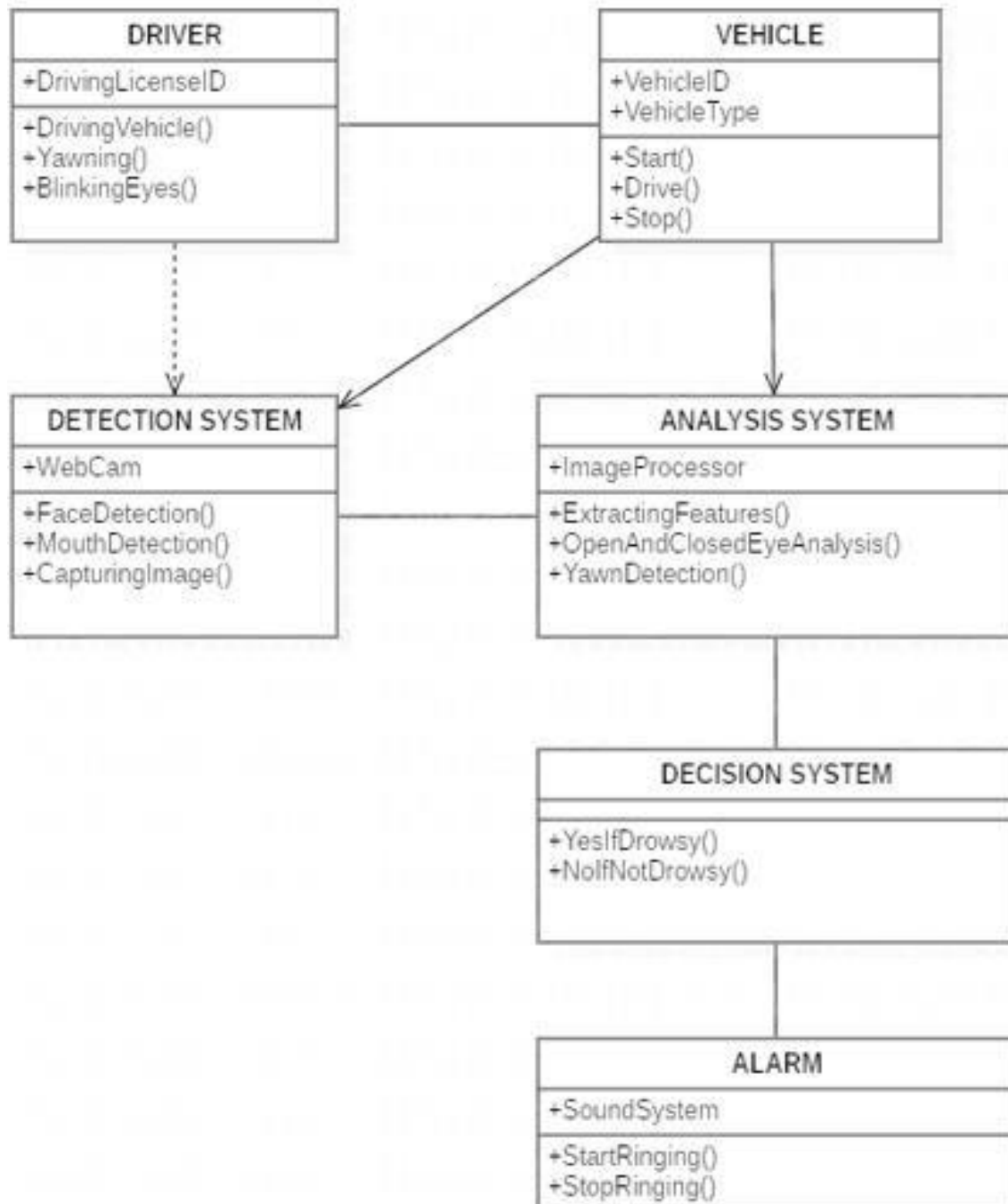
Use Case Diagram

4.2 ACTIVITY DIAGRAM



Activity Diagram

4.3 CLASS DIAGRAM



Class Diagram

CHAPTER 5

SYSTEM TESTING

5.1 OBJECTIVES OF TESTING

Testing is a process of executing a program with the intent of finding an error. A good test case is one that has a high probability of finding an as yet undiscovered error.

A successful test is one that uncovers a yet undiscovered error.

5.2 TESTING STRATEGIES

The extent of testing a system is controlled by many factors, such as the risk involved, limitations on resources, and deadlines. There are many testing strategies, but most used testing combinations are as follows:

- White Box Testing
- Black Box Testing
- Top-Down Testing
- Bottom-Up Testing

5.2.1 White Box Testing

In white box testing the specific logic is important and must be tested to guarantee the system's proper functioning. The main use of white box testing is in error-based testing.

5.2.2 Black Box Testing

The concept of black box is used to represent a system whose inside workings are not available for inspection. In Black box testing the logic is unknown; all that is known is what goes in and what comes out, or the input or output.

5.2.3 Top-Down Testing

In Top-Down testing the main logic or object interactions and system messages of the application need more testing than an individual object's methods or supporting logic.

5.2.4 Bottom-Up Testing

Bottom-Up testing starts with the details of the system and proceeds to higher levels by a progressive aggregation of details until they collectively fit the requirements for the system. This approach is more appropriate for testing the individual objects in a system.

5.2.5 Unit Testing

Unit testing concentrates on each unit of the software as implemented in source code. Unit testing makes heavy use of white box testing techniques, exercising specific path in a module's control structure to ensure complete coverage and maximum error detection.

Test Case	Description	Expected Outcome
Data Preprocessing	Test image loader function with a valid image path	Function successfully loads the image and returns the image data.
Data Preprocessing	Test image loader function with an invalid image path	Function raises an appropriate error.
Data Preprocessing	Test image normalization function with valid image data	Function normalizes the image data to the expected range.
Model	Test model prediction function with valid preprocessed image data	Function generates a caption for the image.
Model	Test model prediction function with invalid preprocessed image data (e.g., wrong dimensions)	Function raises an appropriate error.
Caption Processing	Test function to remove generic terms from caption.	Function removes generic terms while preserving relevant information.

Caption Processing	Test function for grammatical correctness of caption	Function ensures grammatically correct and well-structured caption.
--------------------	--	---

5.2.6 Integration Testing

Integration testing focuses on design and the construction of the software architecture. Integration testing addressed the issues associated with the dual problems of verification and program construction. Developing a strategy for integrating the various web pages of a web site into functioning whole requires careful planning.

Test Case	Description	Expected Outcome
End-to-End Flow	Test complete image caption generation process with a valid image	System successfully loads, preprocesses the image, generates a caption, and performs post-processing.
Error Handling	Test image caption generation process with an invalid image path	System gracefully handles the error and provides a meaningful error message.
Module Interaction	Test interaction between image preprocessor and model	Preprocessed image data is successfully passed to the model for prediction.
Module Interaction	Test interaction between model and caption processing module	Model output is correctly fed into the caption processing module.
Performance	Test overall processing time for image caption generation	System generates captions within an acceptable timeframe.

5.3 TEST CASES AND TEST RESULTS

Test ID	Test Case Title	Test Condition	System Behavior	Expected Result
T01	NSGY	Straight Face, Good Light, With Glasses	Non Drowsy	Non Drowsy
T02	YTGN	Tilted Face, Good Light, No Glasses	Drowsy	Drowsy
T03	YTGy	Tilted Face, Good Light, With Glasses	Drowsy	Drowsy

Note: Testing is performed manually

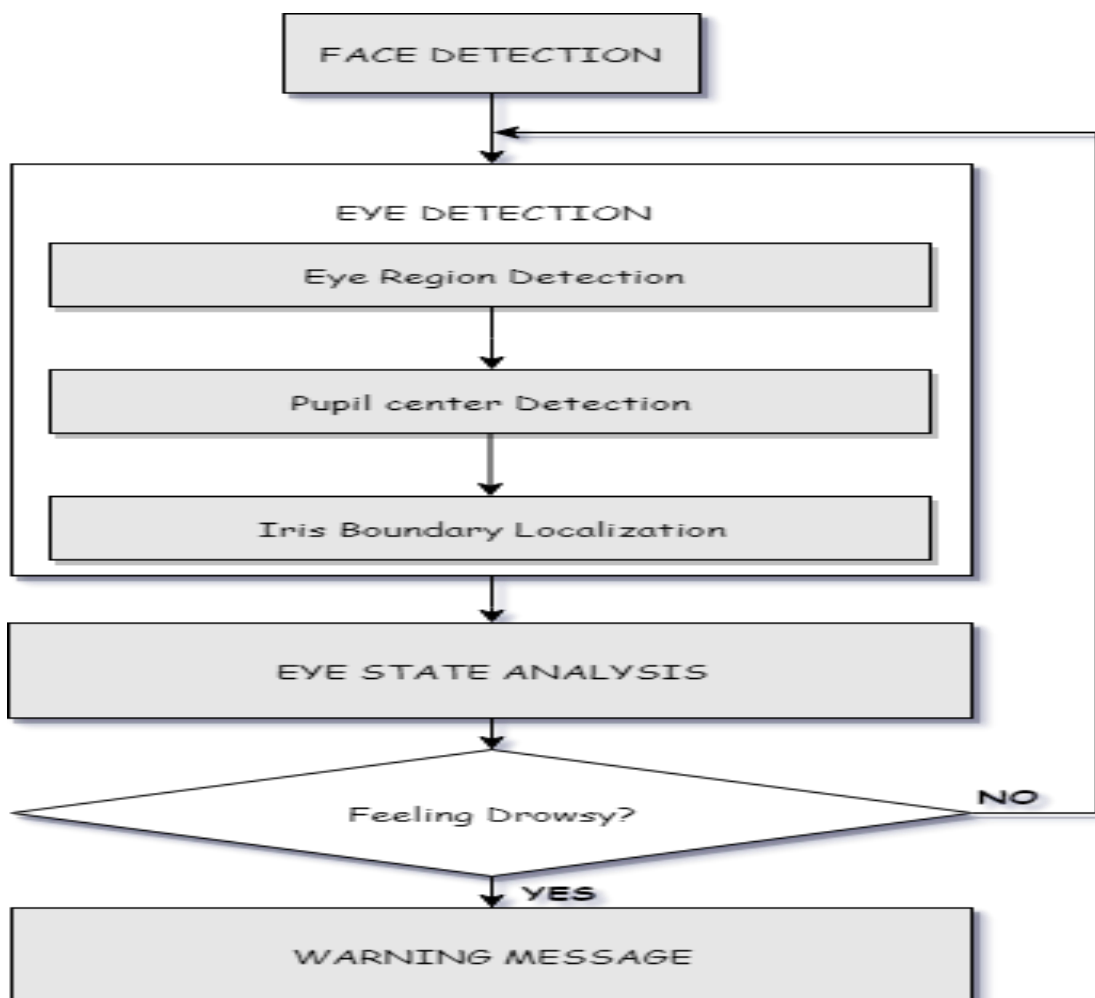
CHAPTER 6

PROJECT PLANNING

6.1 SYSTEM MODEL

The framework is created utilizing the incremental model. The center model of the framework is first created and afterwards augmented in this way in the wake of testing at each turn. The underlying undertaking skeleton was refined into expanding levels of ability.

At the following incremental level, it might incorporate new execution backing and improvement.

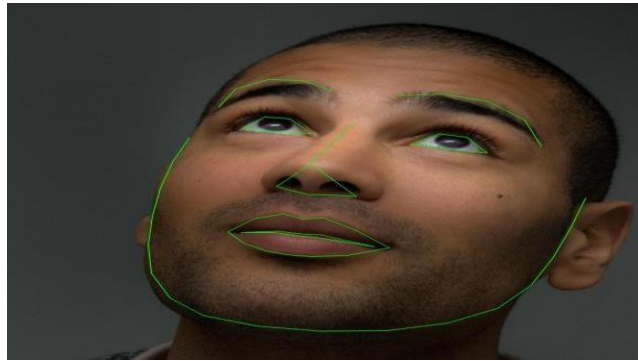


Block Diagram

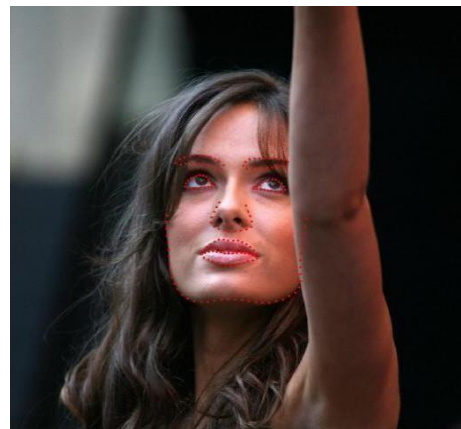
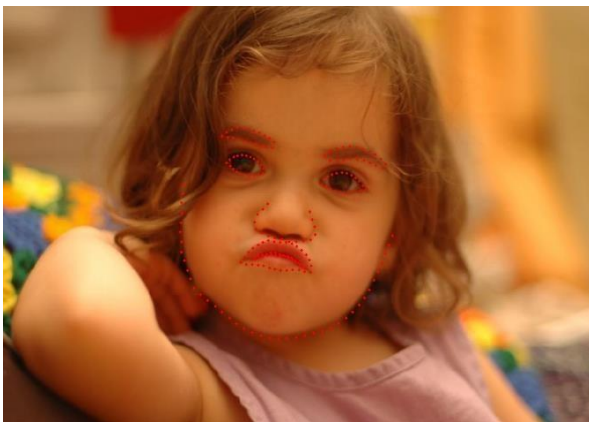
CHAPTER 7

IMPLEMENTATION

In our program we used Dlib, a pre-trained program trained on the HELEN dataset to detect human faces using the pre-defined 68 landmarks.



7.1 Landmarked Image of a person by Dlib

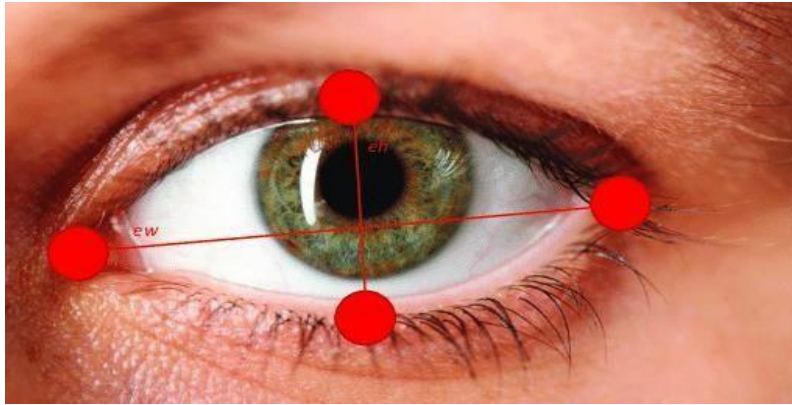


7.2 HELEN Dataset Samples

After passing our video feed to the dlib frame by frame, we are able to detect left eye and right eye features of the face.

Now, we drew contours around it using OpenCV.

Using Scipy's Euclidean function, we calculated sum of both eyes' aspect ratio which is the sum of 2 distinct vertical distances between the eyelids divided by its horizontal distance.



7.3 Eyes With Horizontal And Vertical Distance Marked For Eye Aspect Ratio Calculation.

Now we check if the aspect ratio value is less than 0.25 (0.25 was chosen as a base case after some tests). If it is less an alarm is sounded and user is warned.

CHAPTER 8

SOURCE CODE

PYTHON CODE (IDLE):

```
from scipy.spatial import distance
from imutils import face_utils
import imutils
import dlib
import cv2

def eye_aspect_ratio(eye):
    A = distance.euclidean(eye[1], eye[5])
    B = distance.euclidean(eye[2], eye[4])
    C = distance.euclidean(eye[0], eye[3])
    ear = (A + B) / (2.0 * C)
    return ear

thresh = 0.25
frame_check = 20
detect = dlib.get_frontal_face_detector()
predict = dlib.shape_predictor("models/shape_predictor_68_face_landmarks.dat")# Dat file
is the crux of the code

(lStart, lEnd) = face_utils.FACIAL_LANDMARKS_68_IDXS["left_eye"]
(rStart, rEnd) = face_utils.FACIAL_LANDMARKS_68_IDXS["right_eye"]
cap=cv2.VideoCapture(0)
flag=0
while True:
    ret, frame=cap.read()
```

```

frame = imutils.resize(frame, width=450)

gray = cv2.cvtColor(frame, cv2.COLOR_BGR2GRAY)

subjects = detect(gray, 0)

for subject in subjects:

    shape = predict(gray, subject)

    shape = face_utils.shape_to_np(shape)#converting to NumPy Array

    leftEye = shape[lStart:lEnd]

    rightEye = shape[rStart:rEnd]

    leftEAR = eye_aspect_ratio(leftEye)

    rightEAR = eye_aspect_ratio(rightEye)

    ear = (leftEAR + rightEAR) / 2.0

    leftEyeHull = cv2.convexHull(leftEye)

    rightEyeHull = cv2.convexHull(rightEye)

    cv2.drawContours(frame, [leftEyeHull], -1, (0, 255, 0), 1)

    cv2.drawContours(frame, [rightEyeHull], -1, (0, 255, 0), 1)

    if ear < thresh:

        flag += 1

        print (flag)

        if flag >= frame_check:

            cv2.putText(frame,
"*****ALERT!*****", (10, 30),

                        cv2.FONT_HERSHEY_SIMPLEX, 0.7, (0, 0, 255), 2)

            cv2.putText(frame,
"*****ALERT!*****", (10,325),

                        cv2.FONT_HERSHEY_SIMPLEX, 0.7, (0, 0, 255), 2)

            #print ("Drowsy")

        else:

            flag = 0

    cv2.imshow("Frame", frame)

    key = cv2.waitKey(1) & 0xFF

```

```

        if key == ord("q"):
            break

cv2.destroyAllWindows()

cap.release()

```

JUPYTER NOTEBOOK (ipynb):

```

from scipy.spatial import distance

from imutils import face_utils

import imutils
import dlib
import cv2

def eye_aspect_ratio(eye):

    A = distance.euclidean(eye[1], eye[5])
    B = distance.euclidean(eye[2], eye[4])
    C = distance.euclidean(eye[0], eye[3])

    ear = (A + B) / (2.0 * C)

    return ear

thresh = 0.25

frame_check = 20

detect = dlib.get_frontal_face_detector()

predict = dlib.shape_predictor("shape_predictor_68_face_landmarks.dat")

(lStart, lEnd) = face_utils.FACIAL_LANDMARKS_IDXS["left_eye"]
(rStart, rEnd) = face_utils.FACIAL_LANDMARKS_IDXS["right_eye"]

cap=cv2.VideoCapture(0)

flag=0

while True:

    ret, frame=cap.read()

    frame = imutils.resize(frame, width=450)

    gray = cv2.cvtColor(frame, cv2.COLOR_BGR2GRAY)

```

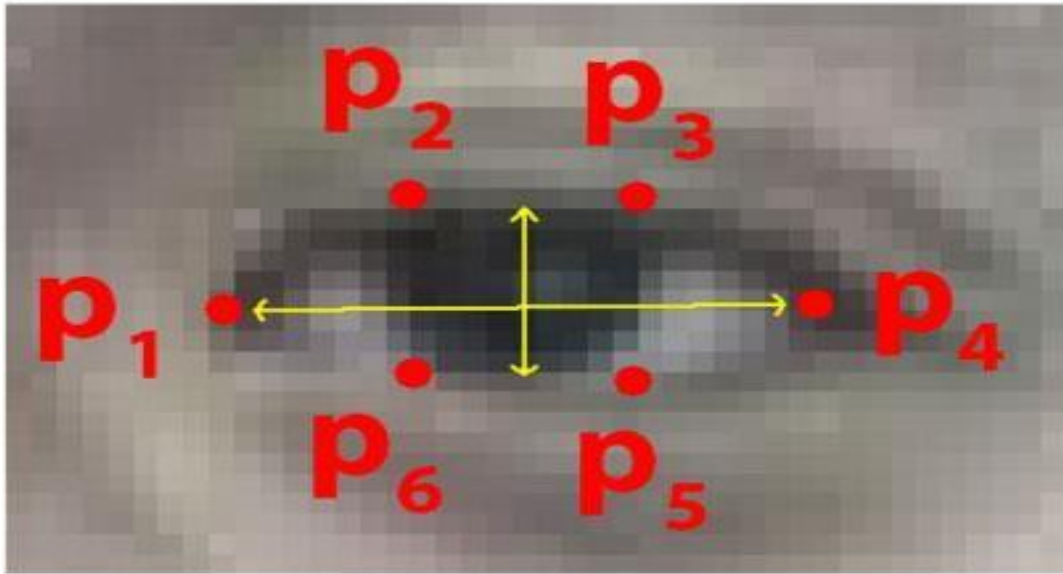


```

subjects = detect(gray, 0)
for subject in subjects:
    shape = predict(gray, subject)
    shape = face_utils.shape_to_np(shape)#converting to NumPy Array
    leftEye = shape[lStart:lEnd]
    rightEye = shape[rStart:rEnd]
    leftEAR = eye_aspect_ratio(leftEye)
    rightEAR = eye_aspect_ratio(rightEye)
    ear = (leftEAR + rightEAR) / 2.0
    leftEyeHull = cv2.convexHull(leftEye)
    rightEyeHull = cv2.convexHull(rightEye)
    cv2.drawContours(frame, [leftEyeHull], -1, (0, 255, 0), 1)
    cv2.drawContours(frame, [rightEyeHull], -1, (0, 255, 0), 1)
    if ear < thresh:
        flag += 1
        #print (flag)
        if flag >= frame_check:
            cv2.putText(frame, "*****ALERT!*****", (10, 30),
                cv2.FONT_HERSHEY_SIMPLEX, 0.7, (0, 0, 255), 2)
            cv2.putText(frame, "*****ALERT!*****", (10,325),
                cv2.FONT_HERSHEY_SIMPLEX, 0.7, (0, 0, 255), 2)
        else:
            flag = 0
    cv2.imshow("Frame", frame)
    key = cv2.waitKey(1) & 0xFF
    if key == ord("q"):
        cv2.destroyAllWindows()
        cap.release()
        break

```

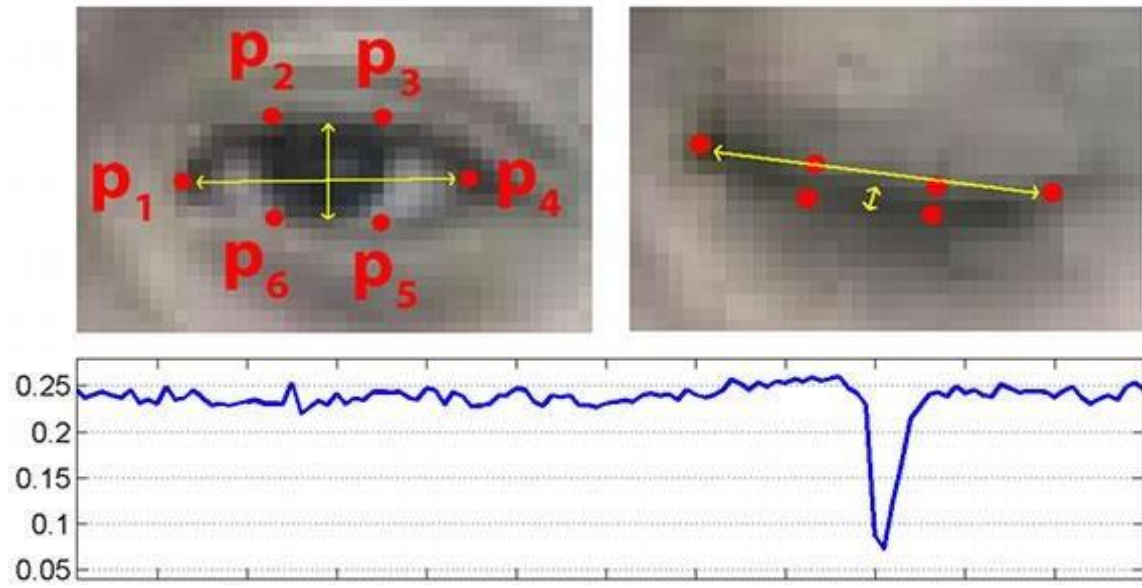
8.1 ASSETS



Eye 1

$$\text{EAR} = \frac{\|p_2 - p_6\| + \|p_3 - p_5\|}{2\|p_1 - p_4\|}$$

Eye 2

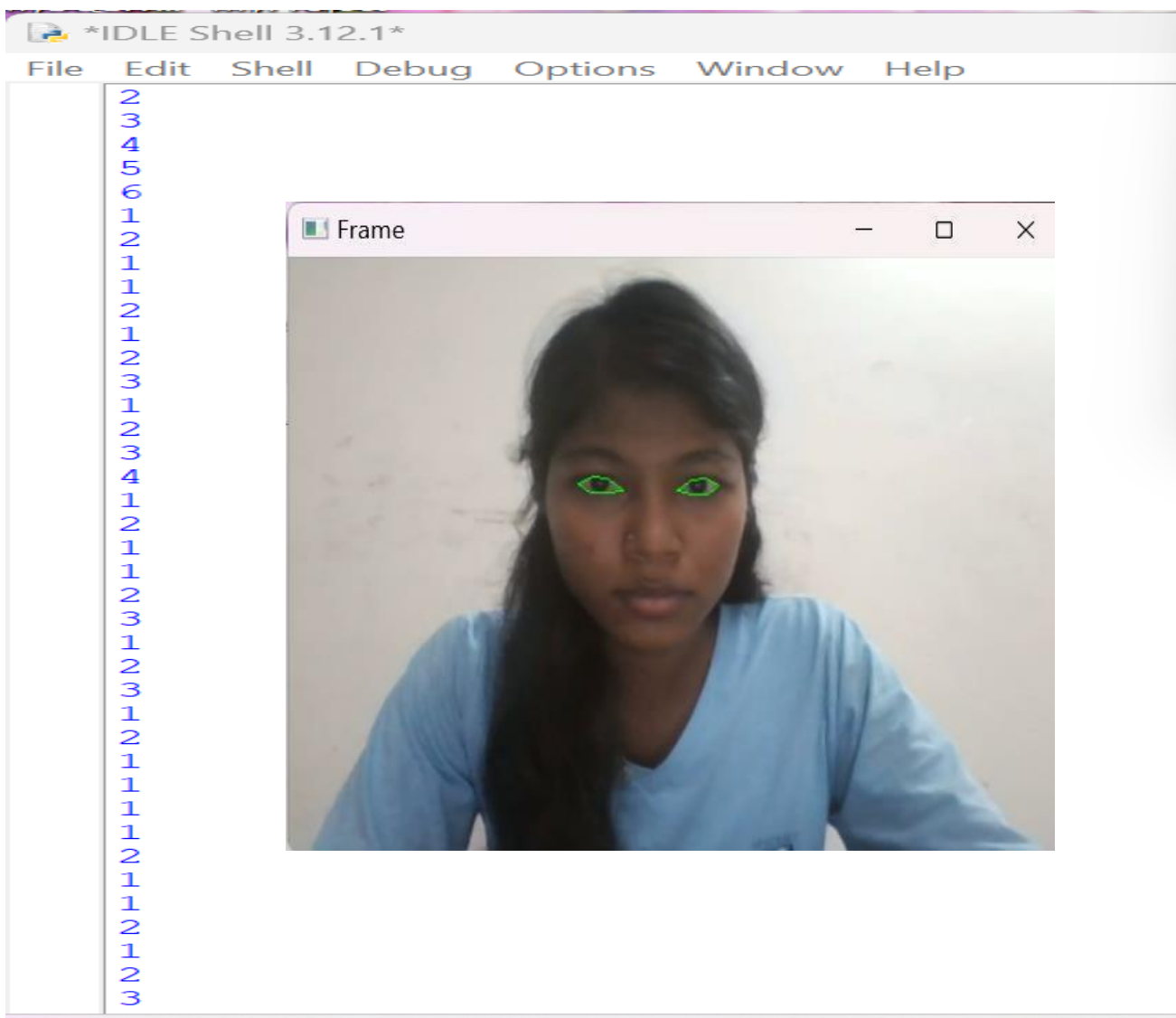


Eye 3

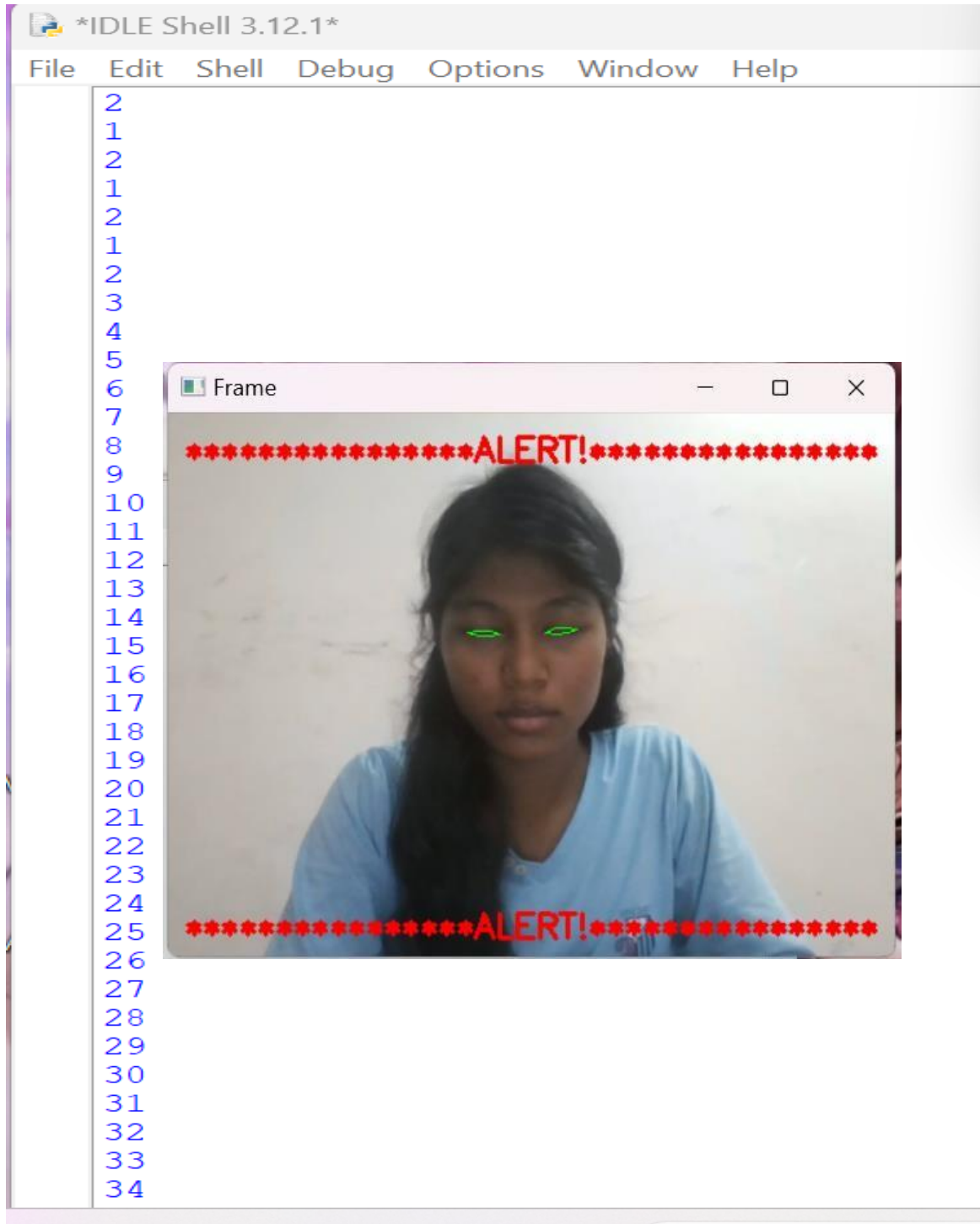
CHAPTER 9

SCREENSHOTS

9.1 Non-drowsy Person:



9.2 Drowsy Person:



CHAPTER 10

CONCLUSION

Our project aim is to address a critical issue of driver fatigue and sleepiness leading to accidents on roads by developing a driver drowsiness detection system. By implementing various techniques, such as utilizing OpenCV for real-time facial recognition and eye state classification, the system provides a practical solution to monitor driver alertness. It completely meets the objectives and requirements of the system. The framework has achieved an unfaltering state where all the bugs have been disposed of. The framework cognizant clients who are familiar with the framework and comprehend it's focal points and the fact that it takes care of the issue of stressing out for individuals having fatigue-related issues to inform them about the drowsiness level while driving.

REFERENCES

- [1] Q. Massoz, T. Langohr, C. Francois, J. G. Verly, "The ULg Multimodality Drowsiness Database" (called DROZY) "2016".
- [2] K. Krafska, A. Khosla, P. Kellnhofer, H. Kannan, "Eye Tracking for Everyone", CVPR, "2016".
- [3] K. Zhang, Z. Zhang, Z. Li, "Joint Face Detection and Alignment using Multi-task Cascaded Convolutional Networks", IEEE Signal Processing Letters, "2016".
- [4] S. Park, F. Pan, S. Kang and C. D. Yoo, "Driver drowsiness detection system based on feature representation learning using various deep networks," ROC, "2016".
- [5] K. Dwivedi, K. Biswaranjan, A. Sethi, "Drowsy Driver Detection using Representation Learning," Advance Computing Conference (IACC), 2014 IEEE International, Feb. "2014".
- [6] Forsman, P.M., Mott, C.G., Dongen, H.P." Efficient driver drowsiness detection at moderate levels of drowsiness" *Accid. Anal. Prev.*, "2013".
- [7] Krizhevsky, A., Sutskever, I., Hinton, G. E.: "Imagenet classification with deep convolutional neural networks", "2012".
- [8] Singh, M., Kaur, G." Drowsy detection on eye blink duration using algorithm", *Int. J. Emerg. Tech. Adv. Eng.* 2, "2012".
- [9] Arun Sahayadhas, "Detecting Driver Drowsiness Based on Sensors", 7 December "2012".
- [10] B. Hariri, S. Abtahi, S. Shirmohammadi, and L. Martel, "A yawning measurement method to detect driver drowsiness," *Distrib. Collab. Virtual Environ. Res. Lab., Univ. Ottawa, Ottawa, ON, Canada*, "2011".
- [11] Z. Mardi, S. N. Ashtiani, and M. Mikaili, "EEG-based drowsiness detection for safe driving using chaotic features and statistical tests," *J. Med. Signals Sens.*, vol. 1, pp. 130–137, "2011".
- [12] Boyle, Paul, A., Rizzo, M, "Driver performance in the moments surrounding a microsleep", *Transp. Res. Part F: Traffic Psychol. Behav.* "2008".
- [13] H. Malik, F. Naeem, Z. Zuberi, and R. ul Haq, "Vision based driving simulation," "2004".
- [14] Smith, P., Shah, M., da Vitoria Lobo, N," Monitoring head/eye motion for driver alertness with one camera", In: *ICPR*, p., "2000".
- [15] Wierwille, W. W., & Ellsworth, L. A., "Evaluation of driver drowsiness by trained raters", *Accident Analysis & Prevention*, 26(5), "1994".
- [16] Yuille, A., Hallinan, P., & Cohen, D., " Feature extraction from faces using deformable templates". *International Journal of Computer Vision*, 8(2), "1992".